

Issues_Summary

Open workable items (current_location = Issues), regenerated from the SQLite single-source-of-truth (§11.4.12).

Counts by Type × Status

Type	Status	Count
Bug	Queued	2
TOTAL		2

Items

ATM ID	Type	Status	Severity	Description
HXC-151	Bug	Queued	Medium	Several regression tests in the server package carry a RED_MODE switch whose job is to prove the test can actually catch the bug it guards. In these cases the RED half checks a private copy of the old code that lives inside the test file itself, instead of the real shipped code. Because that copy is part of the test and never changes when the product changes, the check produces the same result on every build ever made — the working one, the broken one, and any future one — so it can never fail and proves nothing about the product. This matters because everyone reads a green result as evidence that the guard is real, when the guard is in fact inert: the original defect could return and the suite would stay green. The confirmed cases are handlers_systemstatus_guard_test.go (calls a test-local preFixSystemStatusHealthCheck helper

ATM ID	Type	Status	Severity	Description
				instead of the real <code>getSystemStatus</code> handler, so the nil-database guard in the product is bypassed by construction), <code>handlers_qastatus_deadlock_guard_test.go</code> (hand-rolls the locking sequence in <code>reproduceRecursiveRLockDeadlock</code> rather than calling the shipped handler, so it only re-proves a standard Go library property), <code>llm_rag_test.go</code> (deliberately skips the real handler and its retrieval step, so the ‘prompt was not augmented’ result is guaranteed by the test’s own omission), and <code>llm_working_funnel_test.go</code> (one case has no RED branch at all, and another calls a real function that the fix never touched). To reproduce: revert the matching product fix and re-run the test with <code>RED_MODE=1</code>; the branch still passes, which it must not. The work is to rewrite each RED branch so it drives the real shipped code path, exactly as was done for the three sibling guards in <code>llm_default_model_regression_test.go</code> and <code>llm_generate_regression_test.go</code>. Done means that for every listed test, <code>RED_MODE=1</code> passes on an artifact with only its own fix reverted and fails on the current artifact, that this four-way result is captured as evidence, and that a repeat of the enumerated search finds no remaining test-local stand-ins. One test file in the server package chooses between ‘reproduce the old bug’ mode and ‘confirm the fix works’ mode by reading an environment variable, but its default is backwards: when the variable is unset — which is how the suite runs normally and in every automated run — it selects the old-bug mode. The practical effect is that the checks confirming the fixed behaviour never execute at all,
HXC-152	Bug	Queued	High	

ATM ID	Type	Status	Severity	Description
				while the test still reports success, so the team sees a green result for behaviour nobody actually verified. The behaviour left unguarded is the model-listing funnel that is supposed to hide models which failed verification, scored below the quality threshold, or have no API key configured, which is precisely the anti-bluff filtering the original fix was written to provide. A second test in the same file simply skips itself outright under the same default, so the provider key-gate is unguarded too. Every other polarity switch in this package defaults the opposite way, so this file is also inconsistent with its neighbours and will mislead anyone who assumes the shared convention. To reproduce: run the package normally and observe that the guarded assertions are never reached while both tests report PASS. Done means the default is flipped so the fix-verifying assertions run in ordinary runs, the reproduce-the-old-bug mode still works when explicitly requested, and captured evidence shows those assertions genuinely execute and genuinely fail when the filtering is removed.